by the assembler, freeing you from the pain of hand assembly. The assembler supports four types of syntax parts:

- *The assembler Intel 8085 mnemonics*: The instruction strings as in the manuals.
- *Labels:* A named point in the code, the target for JMP or CALL instructions.
- *Comments*: Anything on a line after a semicolon ';' is ignored by the assembler, and used to write comments explaining the code.
- *Pseudo op-codes*: Are not actual op-codes, but instructions to the assembler that provide some features to ease the coding process.

Instead of repeating the assembler features of the simulator here, I recommend you read the official site, where every detail is documented: *http://gnusim8085.org/doc/asm-guide.html*. From version 1.3.7, this *assembler tutorial* is included with the software, available at Help > Assembler Tutorial.

After using this software for quite a long time, I have found some positive and negative points about it, which I will discuss below.

The highs

- Has a detection mechanism for when your code goes wild. For example, if you forget to place a HLT to stop processing, the simulator will automatically stop program execution and warn you about this.
- If you do more POP operations than *PUSH*, the simulator stops execution and tells you to check the logic. Although there is no warning on the real 8085 (it cannot—it would simply take the contents of SP as the top of the stack), this is very helpful, because such unbalanced stack operations are hard to detect in a large program.
- The stack view and tracing is a good feature and is really helpful for debugging code with many subroutines and a lot of *PUSH*, POP and other stack-related instructions.
- Like most simulators, this has a very good and easy-to-read register and flag views, which helps in programming.
- It provides an assembler and lets you enter the program in Intel 8085 mnemonics instead of hand assembly as on the real 8085 (when programmed manually).
- Has debugging features.
- Has assembler listing with address location, op-codes, mnemonics and comments -- all as a single list. This listing can be followed when entering the code manually on the real 8085, and also keep the mnemonics of the op-codes at hand. Good for documentation.
- The editor has good highlighting, which makes the code easier to read
- Most importantly, it has a very simple and clutter-free interface.

The lows

Here are some points that I felt needed improvement:

- The label line needs to be assigned at least a line of code. For example, if you do something like the following:

```
mvi c, 0ah
mvi b, 05h
xra a
loop:
add b
dcr c
jnz loop
```

The above code would not assemble, as the line with the label '*loop*' does not have any code. Either you have to move the ADD B to the '*loop:*' label line, or simply include a NOP beside the label '*loop*'.

- The memory and I/O contents are displayed in decimal, which causes a lot of confusion as the registers' displays are in hex. Thus, each time, you need to convert the values and check the output. Also, because of this difference, if you are working with code that manipulates a good amount of data from memory, it becomes even more difficult to trace, as each time you need to pay attention to the base of the number you are looking at.
- Representation of the I/O, and the memory editing with the spin-box makes memory data entry and inspection very slow. For example, if you enter some data manually in memory, then you need to: click on the text box, enter data, click *Update*... and continue like this. I would recommend that memory and I/O should be editable with the mouse and also with easy keyboard short-cuts. For example, a grid display with scroll-bars showing 10-20 memory locations, navigable with the keyboard, would be extremely helpful.
- The editor has only one tab—so if you open another file when one file is already open, the new replaces the old. An MDI feature would be great, but as it is, this is not much trouble, in my opinion.

Bugs

While using and testing the 8085, I have found some potential problems, bugs and glitches in the simulator, which are listed here.

- The PUSH PSW shows an invalid string for the register name in the *Stack* debugging tab.
- There is a bug related to opening and closing files. When you are working with one file and then open another, the editor displays the new file—but what is not obvious is that the old file (which you were working on) is not closed. This is not much trouble, since the limit for the maximum open files is quite large by default (run *ulimit* -n), but this should be fixed. You can check it by first launching the program, and then keeping an eye on the output of *watch ls -l /proc/$(pidof gnusim8085)/fd*.
- There is a severe problem with the simulator if you